



it's already in my prompt, attached.

also, your job is not to provide research projects but A SOLUTION. if you need to provide multiple solutions because you are not sure which works, that's fine. But you cannot simply make this a research project - that's why you are being asked, since you are part of the research.

You are right: the attachment was present, and I should have read it before replying. I have now used it **only to identify the requirements and measured failure modes**, not as evidence that a proposed remedy works. The solution I would build is a *canonical reconstructed-image codec*: make the legal pixels themselves the stable representation, and make every transmitted decision a deterministic function of those pixels and a specified decoded reference state. That changes where quantisation happens; it does not add a recovery search to the existing wavelet pipeline. ^[1]

There is an important boundary to the claim. I can give you a concrete architecture that guarantees legality, generation stability, bounded work, and an exact-size output under a specified rate envelope. I cannot honestly assert, without implementing and viewing it, that it also matches JPEG XS at half its bitrate with no visible artifact or quality cost. JPEG XS itself targets low latency, precise rate control, and multigeneration robustness; those are meaningful benchmarks, not evidence that this different construction meets your more stringent byte-exact requirement. ^{[2] [3]}

Build the stable image first

Replace transform-cell reconstruction as the authority with a legal, idempotent sample reconstruction. For each component, define a fixed, nested set of *legal* integer sample values for each supported precision level. Quantise an incoming sample to one of those values, with fixed tie-breaking. The finest level contains every legal integer. At any given level, applying the quantiser again to one of its own outputs changes nothing.

Code those reconstructed samples using reversible prediction from already reconstructed neighbours and, where appropriate, the one previous decoded frame. The prediction determines an efficiently coded *residual*, but it must not shift the set of sample values that the encoder is permitted to reconstruct. Otherwise, the next encoder can see the same pixel but place it in a different prediction-relative quantisation cell. Encoder and decoder both reconstruct each pixel immediately, so their causal prediction states agree. Causal reconstructed-neighbour prediction and in-loop quantisation are established low-complexity coding patterns; this particular canonical-pixel construction is my proposed application of them, not a demonstrated OMC-1 result. ^{[4] [5]}

This directly settles the dense black/white-edge case. A legal black or white value is a permissible reconstruction value, not the result of clipping an illegal inverse-transform output.

No clipped excursion has to be inferred, and an arbitrarily long rail edge does not trigger coupled correction sweeps. It also removes the need for the decoder-side in-cell legaliser and for the next encoder’s lattice-point lock. The memo’s results show why neither existing step currently closes that loop. ^[1]

Do not put a lossy inverse wavelet after this canonicalisation. If a transform remains, use it solely as an exactly reversible way of coding differences between the already chosen legal pixels. An exactly invertible integer transform can represent those differences, but invertibility alone does not make *lossily quantised transform coefficients* reconstruct to legal, canonical pixels—the distinction at the heart of the present failure. ^{[6] [1]}

Make decisions reproducible

For a decoded sequence y , the required property is not merely $D(E(y)) = y$; it is $E(D(b)) = b$ for each stream b the encoder emits, with a defined initial state and byte-filling rule. I would impose the following design rules:

Decision	Canonical rule
Reconstructed values	Select from legal, nested sample alphabets; quantising an output again returns that output.
Spatial and temporal predictors	Derive solely from reconstructed pixels and a specified previous decoded frame; use deterministic arithmetic and tie-breaking.
Inter/intra and refresh	Use a fixed, published schedule or a deterministic rule based on decoded state—not an original-picture cost comparison that a later encoder cannot repeat.
Entropy syntax	Use one fixed traversal, context derivation, table choice, termination, and padding convention for a given reconstructed image and state.
Slice boundaries	Reset or explicitly define every dependency at the boundary. Remove a display-side blend that requires a subsequent encoder to undo a hidden pre-display image.
Rate allocation	Make the allocation reproducible from decoded data, or use a fixed allocation. Do not select a plan from source-dependent trial encodes and expect a later encoder to rediscover it.

The **rate-allocation rule** is the hard part—not a detail the pixel quantiser magically solves. A source-dependent decision to give a slice more precision can change after that precision is discarded. If two allowed precision plans reconstruct to the same pixels, the pixels alone cannot tell the next encoder which plan was used. Likewise, if two different valid byte streams decode to the same image and state, a metadata-free next encoder cannot reproduce both. Thus the syntax must admit exactly one encoder-chosen representation per reachable decoded image and state. That is a design constraint, not a job for a 50-pass plan search. ^[1]

For a first implementation, I would choose **fixed, format-and-position-derived slice budgets and precision schedules**, encode causal residuals once, and pad each fixed-size output unit canonically. That makes the allocation reproducible and exact CBR straightforward *provided every unit’s coded data fits*. A short, fixed-list local choice is possible later only if each choice has a disjoint set of attainable decoded images and its choice can be recovered with bounded logic from those images. I would not claim such a choice is free in bits or quality without measuring it.

Bound the work and the rate

The proposed baseline has one causal reconstruction-and-symbol pass and one entropy-pack pass per slice, plus fixed control and padding work. There is no inverse-transform repair loop, candidate-plan enumeration, or density-dependent clipping search. Its per-sample operations can be built around comparisons, shifts, adds, and lookups. That is a credible **bounded-work architecture**, though an actual ZU7EV timing and resource fit still depends on pixel throughput, memory traffic, format, and entropy implementation. JPEG XS demonstrates the relevance of line-scale, rate-controlled processing as a comparison point, not that this proposed design already meets its throughput. ^[7] ^[2]

There is, however, no honest one-pass guarantee that *arbitrary* slice content will fit a prescribed narrow budget while also forbidding any additional quality loss. Fixed budgets and padding guarantee exact pipe occupancy only if you specify an overflow-safe fallback. A fallback to a coarser legal sample alphabet remains stable and bounded-work if selected by a reproducible rule, but it can visibly worsen difficult content. A fallback to exact samples can exceed the budget. The same finite-bit-budget limitation applies to any architecture: some input distinctions must be discarded when there are more distinguishable pictures than available fixed-length codewords. The requirement that this discard **only** soften already-soft regions, on *all* graphics and scene cuts as well as natural pictures, is therefore not something I can guarantee by architectural assertion.

My engineering choice would be to make that limit explicit in the product specification: define the operating bitrate and an admissible source/quality envelope, reserve a worst-case bound for the permitted precision levels, then test the full frame—including first frames, cuts, and dense legal-limit graphics—against the visual criterion. If the stipulated bitrate cannot fund the fallback precision required by that criterion, I would report a failed requirement rather than conceal it behind padding, extra passes, or a nominal “one-pass” rate controller.

Where the current design went wrong

The primary mistake is the **order of operations**: choose lossy transform-domain numbers, invert them into potentially illegal pixels, clip those pixels, then ask another encoder to reconstruct the lost transform-domain decision from the clipped result. The measured sparse and dense failures are different scales of the same information-loss problem. The in-cell legaliser improves reconstruction but still leaves coefficients away from the points expected by the lock; that result is consistent with the architectural diagnosis, not a reason to run a larger lock. ^[1]

The second mistake is treating plan recovery as a substitute for canonical rate control. The first encoder’s source-dependent plan is not necessarily encoded in the decoded image; a faster plan search cannot recover information that is not there. Finally, the cross-slice term and reversible-undo display blend create additional state that must be uniquely reconstructible at a baseband hop. I would remove them from the correctness path and handle slice seams through the coding predictor and reconstruction design instead. ^[1]

Bottom line: build and evaluate the canonical legal-pixel, causal-prediction prototype, with fixed reproducible allocation first. It gives a direct proof route for legality, unlimited-generation

image and byte stability, and bounded per-slice work. Whether it reaches the requested *half-JPEG-XS-rate visual quality* is an empirical pass/fail criterion, not a property I can responsibly promise from first principles or from the attachment.

✱✱

1. MEMO_TO_CODEC_EXPERT_from_Fable_010.md
2. <https://jpeg.org/jpegxs/>
3. https://www.digitalmedia.fraunhofer.de/content/dam/dcinema/en/documents/whitepaper_jpegxs/FHG_JPEG_XS_preprint-IEEE_Broadcast_Technology2019-02.pdf
4. <https://standards.iteh.ai/catalog/standards/iso/08cbcd27-7224-41ec-9ffd-701b48eb9ed1/iso-iec-14495-1-1999>
5. <https://arxiv.org/html/2403.17677v1>
6. <https://www.ni.com/docs/en-US/bundle/labview-advanced-signal-processing-toolkit/page/integer-wavelet-transform.html>
7. <https://www.jpegxs.com/how-jpeg-xs-works---a-simple-introduction>
8. <https://www.rfc-editor.org/info/rfc9134/>
9. <https://www.loc.gov/preservation/digital/formats/fdd/fdd000545.shtml>
10. <https://www.iso.org/obp/ui/en/>
11. <https://arxiv.org/html/1507.04971v1>
12. https://en.wikipedia.org/wiki/JPEG_XS
13. https://repository.arizona.edu/bitstream/handle/10150/675416/ITC_2024_24-09-01.pdf?sequence=1
14. <https://arxiv.org/html/2304.08269v1>
15. <https://waseda.elsevierpure.com/en/publications/learned-image-compression-with-fixed-point-arithmetic/>
16. <https://www.alphaxiv.org/abs/2401.08920>
17. <https://standards.iteh.ai/catalog/standards/iso/d59cb04a-0a9b-4e60-98ca-7d268b78d30e/iso-iec-21122-1-2024>
18. <https://drastic.tv/support-59/supporttipstechnical/233-drastic-jpeg-xs>
19. <https://stackoverflow.com/questions/14841834/is-re-encoding-jpeg-images-an-idempotent-operation>
20. <https://www.rfc-editor.org/rfc/rfc9134.html>
21. <https://ui.adsabs.harvard.edu/abs/arXiv:2410.11025>
22. <https://ieeexplore.ieee.org/iel7/8784029/9314963/09623349.pdf>
23. <https://arxiv.org/html/2410.11025v1>
24. <https://en.wikipedia.org/wiki/Transcoding>
25. <https://arxiv.org/html/2202.07513v2>
26. <https://www.eetimes.com/video-codecs-tutorial-trade-offs-with-h-264-vc-1-and-other-advanced-codecs/>
27. <https://www.fastcompression.com/blog/jpeg-xs-overview.htm>
28. <https://hello-fri-end.github.io/2026/06/integer-quantization-deep-dive/>
29. <https://www.berglogic.com/en/ip-cores/jpeg-xs>
30. <https://cleanor.app/reference/transcoding>
31. <https://pubmed.ncbi.nlm.nih.gov/18249628/>

32. <https://www.nature.com/articles/s41598-026-66039-6>
33. <https://link.springer.com/content/pdf/10.1007/978-81-322-2580-5.pdf>
34. <https://proceedings.spiedigitallibrary.org/proceedings/Download?urlId=10.1117/12.476372>
35. <https://ui.adsabs.harvard.edu/abs/2005ccec.conf..186Z/abstract>
36. <https://web.stanford.edu/class/ee398a/handouts/lectures/06-Prediction.pdf>
37. <https://www.semanticscholar.org/paper/Reversible-integer-to-integer-wavelet-transforms-Adams-Kossentini/c837085109f0ad87a66e28467a3f0be73699fb52>
38. https://en.wikipedia.org/wiki/Lossless_JPEG
39. <https://iconline.ipleiria.pt/bitstreams/e7e9ff7c-489d-45db-ab27-cd647bd6625f/download>
40. <https://www.mdpi.com/1099-4300/28/8/838>
41. <https://cm-bell-labs.github.io/who/wim/papers/icip97.pdf>
42. <https://www.yumpu.com/it/document/view/5683036/greenlatinpdf>
43. https://proceedings.neurips.cc/paper_files/paper/2023/file/2a25d9d873e9ae6d242c62e36f89ee3a-Paper-Conference.pdf
44. <https://dl.acm.org/doi/pdf/10.1145/3588444.3591007>